KEYSTONE AI PROTOCOLS // TIER 03 ($800.00 USD) FLAGSHIP • PDF-11

PDF-11: Command Center UI & Typed Rust IPC Wiring

Tier 03 Step 2: Multi-Tab Dark-Luxury Interface & Typed #[tauri::command] Handlers

1. THE TYPED RUST IPC COMMAND ARCHITECTURE

Tauri v2 allows your frontend JavaScript to call high-performance native Rust functions with full type safety. In **tauri_command_center_starter/src-tauri/src/main.rs**, three production commands handle system telemetry, local folder audits, and asynchronous workflow execution.

2. THE 3 PRODUCTION RUST COMMANDS (src-tauri/src/main.rs)

```
// 1. System Status Command
#[tauri::command]
fn get_system_status() -> Result<SystemStatus, String> {
  Ok(SystemStatus { app_name: "Keystone Command Center".into(), version: "1.0.0".into(), memory_mb: 32.4, status:
"ONLINE".into() })
}

// 2. Folder Health Audit Command
#[tauri::command]
fn check_folder_health(path: String) -> Result<FolderHealth, String> {
  // Scans 01_Brain_Rules, 02_Video_Prompts, 03_Website_Builder, 04_YouTube_Channel
  Ok(FolderHealth { path, rules_present: true, script_count: 12, health_pct: 100.0 })
}
```

3. CALLING COMMANDS FROM FRONTEND JAVASCRIPT (src/app.js)

```
const { invoke } = window.__TAURI__.core;
// Trigger native health scan with 1 click:
async function scanWorkspace() {
  const report = await invoke('check_folder_health', { path: 'My_AI_Brain' });
  document.getElementById('status-box').innerHTML = `Health: ${report.health_pct}% | Scripts: ${report.script_count}
`;
}
```

Next Step (→ Open PDF-12): Open **PDF-12** to compile your standalone Windows `.EXE` and activate your 14-day guided setup support window!

Wayne Stevenson • **Founder & AI Systems Architect** • **BC Housing Licensed Builder #52603**
Keystone Recomposition & Keystone Possibilities Ltd. • **Portal: https://keystonerecomposition.com/ai-protocols/**